Space Shuttle Main Engine

November 11, 2026

Contents

1 Introduction **1**

2 Oxygen **2**
2.1 First Enthalpy Exchanger 2
2.2 Combustion Chamber 4
2.3 Nozzle Performance 7

A Code listings **8**
A.1 SSMEMain.py 8
A.2 exchanger.py 10
A.3 Iterater.py 17

1 Introduction

The Space Shuttle Main Engine (SSME) is a liquid-fueled rocket engine used to power the Space Shuttle Orbiter. This report will discuss the design and methodologies of the SSME code, as well as the results of the code. For this project, we were given the parameters below:

Combustion Chamber Data	
Fuel	Hydrogen
Oxidizer	O_2
Fuel/Oxidizer Ratio	0.166
Hydrogen Injection Temperature	850 K
Oxidizer Injection Temperature	530 K
Chamber Pressure	204 atm

R_t	5.15 inch
R_e	45.35 inch
θ_p	32 degree

2 Oxygen

2.1 First Enthalpy Exchanger

We begin with computing the equivalence ratio ϕ for the oxygen first enthalpy exchanger. The equivalence ratio is defined as the ratio of the mass of fuel to the mass of oxidizer. The mass of fuel is the mass of hydrogen, and the mass of oxidizer is the mass of oxygen. The mass of hydrogen is $\phi \frac{mol_{O_2}}{mol_{H_2}} = \Phi$.

From here we are to calculate the stndard heat of formation by:

$$\Delta\hat{h}_{H_2} = \hat{h}_{H_2}(850) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{H_2} p}{\hat{R}T^2} dp \tag{1}$$

$$\Delta\hat{h}_{O_2} = \hat{h}_{O_2}(530) + \hat{R}T^2 \int_{pf}^{pi} \frac{1}{p} \frac{-\hat{b}_{O_2} p}{\hat{R}T^2} dp \tag{2}$$

Species	$\hat{b}$
H_2	0.0266
O_2	0.0318
H_2O	0.03049
N_2	0.0391
F_2	0.02896
HF	0.0739

Where $\hat{b}$ is read from a table developed in `exchanger.py` file. $\Delta\hat{h}_i$ is calculated from both hydrogen and oxygen. Once we calculate $\Delta\hat{h}_i$ for each, we can calculate the enthalpy of the mixture.

$$\Delta\hat{h}_{mix} = \Phi\Delta\hat{h}_{H_2} + \Delta\hat{h}_{O_2} \tag{3}$$

We can then apply this to our chemical equation.

$$O_2 + \Phi H_2 \rightarrow (2-\Phi)H_2 + 2H_2O \tag{4}$$

Note: notice the $(2-\Phi)$ in the equation above. This is because we calculated the enthalpy of formation for hydrogen and not hydrogen gas H_2.

From here we can sum the molar enthalpies of formation to get the first estimate of the first reaction enthalpy

$$\Delta\hat{h}_r = \Phi\Delta\hat{h}_{H_2} + 2(\Delta\hat{h}_{H_2O}) \tag{5}$$

From here we go into our Temperature finding Algorithm. We are given the following parameters:

$$r = \Phi - 1 \tag{6}$$

$$q = 1 \tag{7}$$

$$r_{percent} = \frac{r}{r+q} \tag{8}$$

$$q_{percent} = \frac{q}{r+q} \tag{9}$$

From here we use the values of (Eq. 6) - (Eq. 9) to calculate the temperature of the first enthalpy exchanger.

$$T_{H_2} = \texttt{h2_tableT}(\Delta\hat{h}_r) \tag{10}$$

$$T_{H_2O} = \texttt{ho2_tableT}(q_{percent}\Delta\hat{h}_r) \tag{11}$$

Once we have returned the values of Temperature, we then average the two values to get the final temperature of the first enthalpy exchanger.

$$T_C = \frac{T_{H_2} + T_{H_2O}}{2} \tag{12}$$

2.2 Combustion Chamber

For the Combustion Chamber, we have the two equations below:

$$H_2O \rightarrow OH + H \tag{13}$$

$$H_2 \rightarrow 2H \tag{14}$$

The Gibbs Free Energy for each species is given by:

$$\hat{g}_{H_2O} = \texttt{h2o_table_g}(T_C) \tag{15}$$

$$\hat{g}_{H_2} = \texttt{h2_table_g}(T_C) \tag{16}$$

$$\hat{g}_{OH} = \texttt{oh_table_g}(T_C) \tag{17}$$

$$\hat{g}_{H} = \texttt{h_table_g}(T_C) \tag{18}$$

We can then calculate the Gibbs Free Energy of the reaction by:

$$\Delta\hat{g}_{H_2} = 2\hat{g}_H - \hat{g}_{H_2} \tag{19}$$

$$\Delta\hat{g}_{H_2O} = \hat{g}_H + \hat{g}_{OH} - \hat{g}_{H_2O} \tag{20}$$

Now is the point where we involve the **Law of Mass Action**:

Species	$\nu'_{(i)}$	$\nu''_{(i)}$	$\nu''_{(i)} - \nu'_{(i)}$
H_2O	1	0	-1
OH	0	1	1
H	0	1	1

We an now use:

$$K_n = \prod_{i=1}^{N_s} X_{(n)}^{(\nu''_{(i)} - \nu'_{(i)})} = (\frac{p}{p_a})^{-\sigma_v} K_p \tag{21}$$

Where;

$$\sigma_v = \sum_{i=1}^{N_s} (\nu''_{(i)} - \nu'_{(i)}) \tag{22}$$

$$K_p = e^{-\frac{\Delta G}{\bar{R}T}} \tag{23}$$

$$\Delta G = \sum_{i=1}^{N_s} (\nu''_{(i)} - \nu'_{(i)}) \hat{g}_{(i)} \tag{24}$$

Both reactions have $\sigma_v = +1$, so we can solve for the equilibrium constant K_n.

$$K_{H_2O} = \left(\frac{204atm}{1atm}\right)^{-1} e^{-\frac{\Delta \hat{g}_{H_2O}}{\bar{R}T_C}} \tag{25}$$

$$K_{H_2} = \left(\frac{204atm}{1atm}\right)^{-1} e^{-\frac{\Delta \hat{g}_{H_2}}{\bar{R}T_C}} \tag{26}$$

Now we move into the chemistry problem.
Atom Balance:

$$H_2 + lO_2 \rightarrow N_{H_2O} H_2O + N_{H_2} H_2 + N_H H + N_{OH} OH \tag{27}$$

Recall, due to the fule being Hydrogen;

$$mass_{fuel} = 2.02 \tag{28}$$

$$mass_{Oxidizer} = \frac{mass_{fuel}}{\phi} \tag{29}$$

$$l = \frac{mass_{Oxidizer}}{2\hat{m}_{O_2}} \tag{30}$$

Where $\hat{m}_{O_2}$ is the molar mass of Oxygen. And ϕ is the Fuel to Oxidizer ratio, and not the equivalence ratio. Revisiting the **Atom Balance** equation, we can now solve for the number of moles of each species.

The atom balances for hydrogen and oxygen are:

$$H: \quad 2 = 2N_{H_2O} + 2N_{H_2} + N_H + N_{OH} \tag{31}$$

$$O: \quad 2l = N_{H_2O} + N_{OH} \tag{32}$$

That is four unknowns with only two equations, so the Law of Mass Action supplies the other two. The mole fractions are normalized by the full total,

$$X_{(i)} = \frac{N_{(i)}}{N_{Total}}, \qquad N_{Total} = N_{H_2O} + N_{H_2} + N_H + N_{OH}, \tag{33}$$

so the two equilibrium relations become

$$K_{H_2O} = \frac{X_H X_{OH}}{X_{H_2O}} = \frac{N_H N_{OH}}{N_{H_2O} N_{Total}} \tag{34}$$

$$K_{H_2} = \frac{X_H^2}{X_{H_2}} = \frac{N_H^2}{N_{H_2} N_{Total}} \tag{35}$$

The four coupled equations (two atom balances and two equilibrium relations) are solved with `scipy.optimize.fsolve` in terms of $\ln N_{(i)}$, which keeps every mole number positive. The starting guess is complete combustion, $N_{H_2O} = 2l$ and $N_{H_2} = 1 - 2l$. At the current chamber temperature the solution is:

$$N_{H_2O} = 0.6844, \quad N_{H_2} = 0.2529, \quad N_H = 0.0488, \quad N_{OH} = 0.0766 \tag{36}$$

Species	$N_{(i)}$	$X_{(i)}$	$m_{(i)}$	$Y_{(i)}$	$\hat{c}_{p(i)}$	$c_{p(i)}$	$Y_{(i)}c_p$
H_2O	N_{H_2O}	$X_{H_2O} = \frac{N_{H_2O}}{N_{Total}}$	$\hat{m}N_{H_2O}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2O)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2O}c_p$
H_2	N_{H_2}	$X_{H_2} = \frac{N_{H_2}}{N_{Total}}$	$\hat{m}N_{H_2}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H_2)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{H_2}c_p$
H	N_H	$X_H = \frac{N_H}{N_{Total}}$	$\hat{m}N_H$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(H)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	Y_Hc_p
OH	N_{OH}	$X_{OH} = \frac{N_{OH}}{N_{Total}}$	$\hat{m}N_{OH}$	$\frac{\hat{m}}{m_{(i)}}$	$\hat{c}_{p(OH)}(T_C)$	$\frac{\hat{c}_p}{\hat{m}}$	$Y_{OH}c_p$

Now we can calculate c_p, we sum the $Y_{(i)}c_p$ values for each species.

$$c_p = \sum_i Y_{(i)}c_p \tag{37}$$

$$c_p = Y_{H_2O}c_p + Y_{H_2}c_p + Y_H c_p + Y_{OH}c_p \tag{38}$$

We perform now calulate the gas constant R.

$$R = \frac{\hat{R}}{m_{Total}} \tag{39}$$

where $m_{Total} = \sum_i X_{(i)}m_{(i)}$ is the mixture molar mass. The mole fractions are normalized by $N_{Total} = N_{H_2O} + N_{H_2} + N_H + N_{OH}$, which exceeds one once dissociation produces H and OH.

$$c_v = c_p - R \tag{40}$$

$$\gamma = \frac{c_p}{c_p - \frac{\hat{R}}{\sum_i X_{(i)}m_{(i)}}} = \frac{c_p}{c_p - \frac{\hat{R}}{m_{Total}}} = \frac{c_p}{c_p - R} = \frac{c_p}{c_v} \tag{41}$$

2.3 Nozzle Performance

We begin by defining the Area Ratio:

$$A_r = \frac{\pi R_e^2}{\pi R_t^2} \tag{42}$$

Allowing for us to solve for the Mach number:

$$A_r = \left(\frac{\gamma+1}{2}\right)^{-\frac{\gamma+1}{2(\gamma-1)}} \frac{1}{M_e}\left[1 + \frac{\gamma-1}{2}M_e^2\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{43}$$

We can solve for Mach number M_e by making use of the `scipy.optimize.fsolve` function, which returns the roots of a non-linear function. After solving for M_e we can calculate the exit velocity v_e, exit Pressure p_e and exit Temperature T_e.

$$T_e = T_c\left[1 + \frac{\gamma-1}{2}M_e^2\right]^{-1} \tag{44}$$

$$p_e = p_c \left[1 + \frac{\gamma - 1}{2} M_e^2\right]^{-\frac{\gamma}{\gamma-1}} \tag{45}$$

$$v_e = M_e \sqrt{\gamma R T_e} \tag{46}$$

After solving for (Eq. 46), (Eq. 45) and (Eq. 44) we can calculate the mass flow rate $\dot{m}$.

$$\dot{m} = \frac{A_t p_c \sqrt{\gamma}}{\sqrt{RT_c}} \left[\frac{2}{\gamma + 1}\right]^{\frac{\gamma+1}{2(\gamma-1)}} \tag{47}$$

Then once you have the mass flow rate you can calculate the thrust $\mathcal{T}$, specific impulse I_{sp} and the thrust coefficient C_T.

$$\mathcal{T} = \dot{m} v_e + (p_e - p_a) A_e \tag{48}$$

where p_a is the ambient pressure ($p_a = 0$ in vacuum, $p_a = 101325$ Pa at sea level).

$$I_{sp} = \frac{\mathcal{T}}{\dot{m} g_0} \tag{49}$$

$$C_T = \frac{\mathcal{T}}{p_c A_t} \tag{50}$$

A Code listings

A.1 SSMEMain.py

```
import pandas as pd
from exchanger import *


if __name__ == '__main__':

    p_starting = 204 #starting pressure in atm
    p_starting = p_starting * 101325 #convert to pascals
    FuelOxidizer_ratio = 0.166
    mol_O2 = 31.998 # g/mol #single molecule mass
    mol_H2 = 2.016 # g/mol #single molecule mass
    phi = (FuelOxidizer_ratio* mol_O2 / mol_H2)/2
```

```
    Hydr_inject_temp = 850 #K
    Ox_inject_temp = 530   #K
    ThroatRadius = 5.15 #inches
    ExitRadius = 45.35 #inches
    FEE_T, FEE_del_h_total, del_h2_hat, b_hat_table = FEE(Hydr_inject_temp, p_starting, Ox_inject_temp, phi)
    print('phi = ', phi)

    print('Temperature of the first enthalpy exchanger = ', FEE_T, 'K')
    print('Total enthalpy change in the first enthalpy exchanger = ', FEE_del_h_total, 'J')


    gamma, R, cp_total, cv, LawOfMassAction_df = OCC(FEE_T, p_starting, FuelOxidizer_ratio)
    print('gamma = ', gamma)

    p_sea_level = 101325 # Pa
    m_dotOx, v_eOx, p_eOx, ThrustOx, c_TOx, I_spOx, T_e_Ox, M_eOx = OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R, p_starting)
    _, _, _, ThrustOx_SL, c_TOx_SL, I_spOx_SL, _, _ = OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R, p_starting, p_sea_level)

    print('Ox exit Mach number = ', M_eOx)
    print('Ox mass flow rate = ', m_dotOx, 'kg/s')
    print('Ox exit velocity = ', v_eOx, 'm/s')
    print('Ox exit pressure = ', p_eOx, 'Pa')
    print('Ox exit temperature = ', T_e_Ox, 'K')
    print('Ox thrust (vacuum) = ', ThrustOx, 'N')
    print('Ox thrust coefficient (vacuum) = ', c_TOx)
    print('Ox specific impulse (vacuum) = ', I_spOx, 's')
    print('Ox thrust (sea level) = ', ThrustOx_SL, 'N')
    print('Ox thrust coefficient (sea level) = ', c_TOx_SL)
    print('Ox specific impulse (sea level) = ', I_spOx_SL, 's')


    print('Flourine Problem')
    b_hat_table, FEE_T_flourine = FEE_F(phi, p_starting, del_h2_hat, b_hat_table, Ox_inject_temp)

    print('Temperature of the first enthalpy exchanger (
```

```
    Flourine) = ', FEE_T_flourine, 'K')
48
49      FCC(FEE_T_flourine)
```

A.2 exchanger.py

```
1  from Iterater import *
2  from scipy.optimize import fsolve
3  import pandas as pd
4  import numpy as np
5
6
7  def FEE(Hydr_inject_temp, p_starting, Ox_inject_temp, phi):
8      '''
9      This is a function that returns the calculations of the
       first enthalpy exchanger
10     :param Hydr_inject_temp: temperature of hydrogen
       injection in K
11     :param p_starting: starting pressure in Pa
12     :param Ox_inject_temp: temperature of oxygen injection in
       K
13     :param phi: equivalence ratio
14     :return T the temperature of the first enthalpy exchanger
15     :return del_h_total the total enthalpy change in the
       first enthalpy exchanger
16     :return b_hat_table the table of b_hat values
17     :return del_h2_hat the enthalpy change of H2
18     '''
19
20
21     h_f_h2o = abs(h2o_table_f(298)) #J/mol
22     h_f_h = h_table_f(298)/1000
23     h_oh_f = oh_table_f(298)/1000  # KJ/mol
24
25     # Van der Waals co-volume b_hat (L/mol)
26     b_hat_table = pd.DataFrame({'substance': ['H2', 'H2O', '
       O2', 'N2'],
27                                 'b_hat': [0.0266, 0.03049,
       0.0318, 0.0391]})
28
29     b_hat_h2 = b_hat_table['b_hat'][0]
30     b_hat_o2 = b_hat_table['b_hat'][2]
31
32     del_h2_hat = h2_tableH(Hydr_inject_temp) + b_hat_h2*(
       p_starting - 101325)/1000
```

```
    del_o2_hat = o2_tableH(Ox_inject_temp) + b_hat_o2*(p_starting - 101325)/1000
    mols_h2 = phi*2
    mols_o2 = 1

    del_h_total = del_h2_hat*mols_h2 + del_o2_hat*mols_o2
    # At this point the chemical equation is O2 + mols*H2 -> (phi-2)*H2 +2H2O
    del_h_total = del_h_total + 2*h_f_h2o

    # weighting function
    r = 0.17947269916082
    q = (1- r)/2

    T_h2o = h2o_tableT(q*del_h_total)
    T_h2 = h2_tableT(r*del_h_total)

    T = (T_h2o + T_h2)/2

    return T, del_h_total, del_h2_hat, b_hat_table

def OCC(FEE_T, p_starting, FO_ratio):
    '''
    This is a function that returns the calculations of the oxygen combustion chamber
    :param FEE_T: temperature of the first enthalpy exchanger in K
    :param p_starting: starting pressure in Pa
    :param FO_ratio: fuel to oxidizer ratio
    :return gamma: the ratio of specific heats
    :return R: the gas constant
    :return cp_total: the total specific heat capacity
    :return cv : the specific heat capacity at constant volume
    :return LawOfMassAction_df: the dataframe of the law of mass action
    '''

    # The chemical equation is H2O -> H + OH
    molfrac_h2o_react = 1
    molfrac_h_react = 0
    molfrac_oh_react = 0
    molfrac_h2o_prod = 0
    molfrac_h_prod = 1
    molfrac_oh_prod = 1
```

```
72
73      # The chemical equation is H2 -> 2H
74      molfrac_h2_react = 1
75      molfrac_2h_react = 0
76      molfrac_2h_prod = 2
77      molfrac_h2_prod = 0
78
79      molfrac_h2o_total = molfrac_h2o_prod - molfrac_h2o_react
80      molfrac_h_total = molfrac_h_prod - molfrac_h_react
81      molfrac_oh_total = molfrac_oh_prod - molfrac_oh_react
82      sigma_nu = molfrac_h2o_total + molfrac_h_total +
        molfrac_oh_total
83      # print('sigma_nu = ', sigma_nu)
84
85      molfrac_h2_total = molfrac_h2_prod - molfrac_h2_react
86      molfrac_2h_total = molfrac_2h_prod - molfrac_2h_react
87      sigma_nu = molfrac_h2_total + molfrac_2h_total
88      # print('sigma_nu = ', sigma_nu)
89
90      # Gibbs free energy of formation
91      g_f_h2o = h2o_table_g(FEE_T)
92      g_f_h = h_table_g(FEE_T)
93      g_f_oh = oh_table_g(FEE_T)
94      g_f_h2 = h2_table_g(FEE_T)
95
96      # Gibbs free energy of reaction: dG = sum_i (nu''_(i) -
        nu'_(i)) g_f(i), i.e. products minus reactants
97      g_r_h2 = 2*g_f_h - g_f_h2              # H2  -> 2H
98      g_r_h2o = g_f_h + g_f_oh - g_f_h2o     # H2O -> H + OH
99
100     k_h2 = ((p_starting/101325)**-1)*np.exp(-g_r_h2/(8.314*
        FEE_T))
101     k_h2o = ((p_starting/101325)**-1)*np.exp(-g_r_h2o/(8.314*
        FEE_T))
102
103
104     # k_h2o = 0.007149
105     # k_h2 = 0.011874839
106
107     mass_fuel = 2.02 #H2
108     mass_oxider = mass_fuel/FO_ratio
109     l = mass_oxider/ (2*15.99)
110
111     # Products per mole of H2 fed: H2 + l O2 -> N_H2O H2O +
        N_H2 H2 + N_H H + N_OH OH
```

```
    #   H atoms:  2N_H2O + 2N_H2 + N_H + N_OH = 2
    #   O atoms:   N_H2O + N_OH               = 2l
    #   H2O -> H + OH:  X_H X_OH / X_H2O = N_H N_OH / (N_H2O N_total) = k_h2o
    #   H2  -> 2H:      X_H^2 / X_H2     = N_H^2   / (N_H2 N_total)   = k_h2
    # Solved in log(N) so every mole number stays positive (no root picking needed).
    def residuals(log_n):
        n_h2o, n_h2, n_h, n_oh = np.exp(log_n)
        n_tot = n_h2o + n_h2 + n_h + n_oh
        return [2*n_h2o + 2*n_h2 + n_h + n_oh - 2,
                n_h2o + n_oh - 2*l,
                np.log(n_h*n_oh/(n_h2o*n_tot)) - np.log(k_h2o),
                np.log(n_h**2/(n_h2*n_tot)) - np.log(k_h2)]

    # start from complete combustion with a little dissociation
    guess = np.log([2*l, 1 - 2*l, 0.01, 0.01])
    log_n, info, ier, msg = fsolve(residuals, guess, full_output=True, xtol=1e-12)
    if ier != 1 or max(abs(r) for r in residuals(log_n)) > 1e-9:
        raise RuntimeError(f'OCC equilibrium solve did not converge: {msg}')

    N_h2o, N_h2, N_h, N_oh = (float(n) for n in np.exp(log_n))
    N_total = N_h + N_oh + N_h2 + N_h2o

    molar_mass_h2o = 18.01528
    molar_mass_h = 1.00794
    molar_mass_oh = 17.00734
    molar_mass_h2 = 2.01588

    LawOfMassAction_df = pd.DataFrame({'substance': ['N_H2O', 'N_H2', 'N_H', 'N_OH', 'Total'],
                                       'mols': [N_h2o, N_h2, N_h, N_oh, N_total]})
    # Mole fractions must be normalized by the total moles: once H and OH form,
    # N_total = N_H2O + N_H2 + N_H + N_OH > 1, so X_(i) != N_(i).
    LawOfMassAction_df['X_(i)'] = LawOfMassAction_df['mols']
```

```
    / N_total
144     # X_(i)*m_hat_(i): the species' contribution to the
    mixture molar mass (g/mol)
145     LawOfMassAction_df['m_hat_(i)'] = pd.Series([
    molar_mass_h2o, molar_mass_h2, molar_mass_h, molar_mass_oh
    , 0.0])\
146                                         * LawOfMassAction_df['
    X_(i)']
147
148     # mixture molar mass m_hat = sum_i X_(i) m_hat_(i), over
    the four species
149     m_hat_total = sum(LawOfMassAction_df['m_hat_(i)'][0:4])
150     LawOfMassAction_df.loc[4, 'm_hat_(i)'] = m_hat_total
151     # mass fractions Y_(i) = X_(i) m_hat_(i) / m_hat
152     LawOfMassAction_df['Y_(i)'] = LawOfMassAction_df['m_hat_(
    i)'] / m_hat_total
153
154     h2_cp = h2_table_cp(FEE_T)
155     h2o_cp = h2o_table_cp(FEE_T)
156     h_cp = h_table_cp(FEE_T)
157     oh_cp = oh_table_cp(FEE_T)
158
159     cp_hats = pd.Series([h2o_cp, h2_cp, h_cp, oh_cp])
160     cps = cp_hats / pd.Series([molar_mass_h2o, molar_mass_h2,
     molar_mass_h, molar_mass_oh])
161     cp_df = pd.DataFrame({'cp_hat': cp_hats, 'cp': cps})
162     cp_df['Y_(i)*cp_(i)'] = cp_df['cp'] * LawOfMassAction_df[
    'Y_(i)'][0:4]
163
164     cp_total = sum(cp_df['Y_(i)*cp_(i)'])
165
166     LawOfMassAction_df = pd.concat([LawOfMassAction_df, cp_df
    ], axis=1)
167
168     R = 8.314/m_hat_total
169     cv = cp_total - R
170
171     gamma = cp_total/cv
172
173     return gamma, R, cp_total, cv, LawOfMassAction_df
174
175 def FEE_F(phi, p_starting, del_h2_hat, b_hat_table,
    Ox_inject_temp):
176     '''
177     This is a function that returns the calculations of the
```

```
    fuel enthalpy exchanger - flourine
    :param phi the equivalence ratio
    :param p_starting combustion pressure in Pa
    :param del_h2_hat heat of formation of H2 from FEE -
    Oxygen
    :param b_hat_table Van der Waals table of constants
    :param Ox_inject_temp: Oxidizer injection temperature in
    K
    :return T the temperature of the FEE in K
    :return b_hat_table the Van der Waals table of constants
    - edited
    '''

    b_hat_table = pd.concat([b_hat_table,
                             pd.DataFrame({'substance': ['F2'
    , 'HF'], 'b_hat': [0.02896, 0.0739]})],
                            ignore_index=True)

    b_hat_f2 = b_hat_table['b_hat'][3]
    F2_inject_temp = Ox_inject_temp

    del_f2_hat = f2_tableH(F2_inject_temp) + b_hat_f2*(
    p_starting - 101325)/1000
    del_hf_hat = abs(hf_tableH(298))
    del_h_total = phi*del_h2_hat + del_f2_hat
    del_h_total = del_h_total + del_hf_hat


    r = phi - 1
    q = 1

    r_perc = r/(r+q)
    q_perc = 1-r_perc

    T_h2 = h2_tableT(del_h_total)
    T_hf = hf_tableT(q_perc*del_h_total)

    T = (T_hf + T_h2) / 2


    return b_hat_table, T

def OxNozzlePerf(ThroatRadius, ExitRadius, gamma, FEE_T, R,
    p_starting, p_ambient=0.0):
    '''
```

```
    This is a function that returns the performance of the oxidizer nozzle
    :param ThroatRadius: throat radius in inches
    :param ExitRadius: exit radius in inches
    :param gamma: ratio of specific heats (frozen, from OCC)
    :param FEE_T: chamber temperature in K
    :param R: gas constant in kJ/kg-K
    :param p_starting: chamber pressure in Pa
    :param p_ambient: ambient pressure in Pa (0 = vacuum, 101325 = sea level)
    :return: m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e
    '''

    Area_ratio = (ExitRadius/ThroatRadius)**2
    # change np.float64 values to a float
    gamma = float(gamma)
    FEE_T = float(FEE_T)
    R = float(R)
    R = R*1000 # convert to J/kg-K

    M_e = float(MachSolve(gamma, Area_ratio, 5.0)[0])
    T_e = FEE_T*((1+((gamma-1)/2)*M_e**2)**(-1))
    v_e = M_e * np.sqrt(gamma * R * T_e)
    p_e = p_starting * (1 + ((gamma - 1) / 2) * M_e ** 2) ** (-gamma / (gamma - 1))
    ExitRadius = ExitRadius/39.37 # convert to meters
    ThroatRadius = ThroatRadius/39.37 # convert to


    m_dot = (p_starting * np.pi * (ThroatRadius)**2) / (np.sqrt(FEE_T)) * \
        np.sqrt(gamma/R)*((2/(gamma+1))**((gamma+1)/(2*(gamma-1))))

    A_e = np.pi * ExitRadius**2
    A_t = np.pi * ThroatRadius**2

    # Control-volume momentum balance: F = m_dot*v_e + (p_e - p_a)*A_e.
    # Chamber pressure only enters through c_T's normalization (p_c*A_t), not the thrust itself.
    Thrust = m_dot * v_e + (p_e - p_ambient) * A_e
    c_T = Thrust / (p_starting * A_t)
    I_sp = Thrust / (m_dot * 9.81)

```

```
254     return m_dot, v_e, p_e, Thrust, c_T, I_sp, T_e, M_e
255 
256 def FCC(FEE_F):
257     '''
258     This is a function that returns the performance of the
    flourine combustion chamber
259     :return:
260     '''
261 
262 
263     # The chemical equation is H2 +F2-> 2HF
264     molfrac_h2_react = 1
265     molfrac_f2_react = 1
266     molfrac_hf_react = 0
267     molfrac_h2_prod = 0
268     molfrac_f2_prod = 0
269     molfrac_hf_prod = 2
270 
271 
272     molfrac_h2_total = molfrac_h2_react - molfrac_h2_prod
273     molfrac_f2_total = molfrac_f2_react - molfrac_f2_prod
274     molfrac_hf_total = molfrac_hf_react - molfrac_hf_prod
275     sigma_nu = molfrac_hf_total + molfrac_h2_total +
    molfrac_f2_total
276     # print('sigma_nu = ', sigma_nu)
277 
278     # Gibbs free energy of formation of H2, F2, and HF
279     del_h_h2 = h2_tableH(FEE_F)
280     del_h_f2 = f2_tableH(FEE_F)
281     del_h_hf = hf_tableH(FEE_F)
282 
283     s_h2 = h2_tableS(FEE_F)
284     s_f2 = f2_tableS(FEE_F)
285     s_hf = hf_tableS(FEE_F)
286 
287     # Gibbs
288     del_g_h2 = del_h_h2 - FEE_F*s_h2
289     del_g_f2 = del_h_f2 - FEE_F*s_f2
290     del_g_hf = del_h_hf - FEE_F*s_hf
291 
292     del_g_total = -del_g_f2 - del_g_h2 + del_g_hf
293     print('del_g_total = ', del_g_total)
```

A.3 Iterater.py

```python
from pathlib import Path

import numpy as np
import pandas as pd
from scipy.optimize import fsolve

# Thermo tables live next to this file, so the code runs from any working directory
TABLES = Path(__file__).resolve().parent / 'tables'


def h2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_closest = h2_table['T'][idx]
    h_hat_close = h2_table['H*-H*_298'][idx]
    if Temp > T_closest:
        T_upper = h2_table['T'][idx+1]
        h_hat_upper = h2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_closest)*(Temp-T_closest)
    else:
        T_lower = h2_table['T'][idx-1]
        h_hat_lower = h2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_closest)*(Temp-T_closest)
    h_hat = h_hat*4184

    return h_hat

def f2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')

    idx = (f2_table['T']-Temp).abs().idxmin()
```

```
    T_cloest = f2_table['T'][idx]
    h_hat_close = f2_table['H*-H*_298'][idx]
    if Temp > T_cloest:
        T_upper = f2_table['T'][idx+1]
        h_hat_upper = f2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = f2_table['T'][idx-1]
        h_hat_lower = f2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)
    h_hat = h_hat*4184

    return h_hat

def o2_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of formation
    '''
    o2_table = pd.read_csv(TABLES / 'o2_table.csv')

    idx = (o2_table['T']-Temp).abs().idxmin()
    T_cloest = o2_table['T'][idx]
    h_hat_close = o2_table['H*-H*_298'][idx]
    if Temp > T_cloest:
        T_upper = o2_table['T'][idx+1]
        h_hat_upper = o2_table['H*-H*_298'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = o2_table['T'][idx-1]
        h_hat_lower = o2_table['H*-H*_298'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)
    h_hat = h_hat

    return h_hat

def hf_tableH(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of
```

```
    formation
    '''
    hf_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')

    idx = (hf_table['T']-Temp).abs().idxmin()
    T_cloest = hf_table['T'][idx]
    h_hat_close = hf_table['h_hat'][idx]
    if Temp > T_cloest:
        T_upper = hf_table['T'][idx+1]
        h_hat_upper = hf_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = hf_table['T'][idx-1]
        h_hat_lower = hf_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
    h_hat = h_hat*4184

    return h_hat

def h2o_table_f(Temp):
    '''
    :param Temp: temperature in K
    :return h_hat: enthalpy in J/mol also known as heat of
    formation
    '''
    h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')

    idx = (h2o_table['T']-Temp).abs().idxmin()
    T_cloest = h2o_table['T'][idx]
    h_hat_close = h2o_table['h_hat'][idx]
    if Temp > T_cloest:
        T_upper = h2o_table['T'][idx+1]
        h_hat_upper = h2o_table['h_hat'][idx+1]
        h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h2o_table['T'][idx-1]
        h_hat_lower = h2o_table['h_hat'][idx-1]
        h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)

    return h_hat
```

```
119
120 def oh_table_f(Temp):
121     '''
122     :param Temp: temperature in K
123     :return h_hat: enthalpy in J/mol also known as heat of
    formation
124     '''
125     oh_table = pd.read_csv(TABLES / 'OH_table.csv')
126
127     idx = (oh_table['T']-Temp).abs().idxmin()
128     T_closest = oh_table['T'][idx]
129     h_hat_close = oh_table['h_hat'][idx]
130     if Temp > T_closest:
131         T_upper = oh_table['T'][idx+1]
132         h_hat_upper = oh_table['h_hat'][idx+1]
133         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
134     else:
135         T_lower = oh_table['T'][idx-1]
136         h_hat_lower = oh_table['h_hat'][idx-1]
137         h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
    T_lower-T_closest)*(Temp-T_closest)
138
139     return h_hat
140
141 def h_table_f(Temp):
142     '''
143     :param Temp: temperature in K
144     :return h_hat: enthalpy in J/mol also known as heat of
    formation
145     '''
146     h_table = pd.read_csv(TABLES / 'H_table.csv')
147
148     idx = (h_table['T']-Temp).abs().idxmin()
149     T_closest = h_table['T'][idx]
150     h_hat_close = h_table['h_hat'][idx]
151     if Temp > T_closest:
152         T_upper = h_table['T'][idx+1]
153         h_hat_upper = h_table['h_hat'][idx+1]
154         h_hat = h_hat_close + (h_hat_upper-h_hat_close)/(
    T_upper-T_closest)*(Temp-T_closest)
155     else:
156         T_lower = h_table['T'][idx-1]
157         h_hat_lower = h_table['h_hat'][idx-1]
158         h_hat = h_hat_close + (h_hat_lower-h_hat_close)/(
```

```
    T_lower-T_closest)*(Temp-T_closest)

    return h_hat

def h2o_tableT(h_hat):
    '''
    :param h_hat: enthalpy in J/mol also known as heat of formation
    :return Temp: temperature in K
    '''
    h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')

    idx = (h2o_table['h_f']-h_hat).abs().idxmin()
    h_hat_closest = h2o_table['h_f'][idx]
    T_close = h2o_table['T'][idx]
    if h_hat > h_hat_closest:
        h_hat_upper = h2o_table['h_f'][idx+1]
        T_upper = h2o_table['T'][idx+1]
        T = T_close + (T_upper-T_close)/(h_hat_upper-h_hat_closest)*(h_hat-h_hat_closest)
    else:
        h_hat_lower = h2o_table['h_f'][idx-1]
        T_lower = h2o_table['T'][idx-1]
        T = T_close + (T_lower-T_close)/(h_hat_lower-h_hat_closest)*(h_hat-h_hat_closest)

    return T

def h2_tableT(h_hat):
    '''
    :param h_hat: enthalpy in J/mol also known as heat of formation
    :return Temp: temperature in K
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
    h_hat = h_hat/4184
    idx = (h2_table['H*-H*_298']-h_hat).abs().idxmin()
    h_hat_closest = h2_table['H*-H*_298'][idx]
    T_close = h2_table['T'][idx]
    if h_hat > h_hat_closest:
        h_hat_upper = h2_table['H*-H*_298'][idx+1]
        T_upper = h2_table['T'][idx+1]
        T = T_close + (T_upper-T_close)/(h_hat_upper-h_hat_closest)*(h_hat-h_hat_closest)
    else:
```

```
198         h_hat_lower = h2_table['H*-H*_298'][idx-1]
199         T_lower = h2_table['T'][idx-1]
200         T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_closest)*(h_hat-h_hat_closest)
201 
202     return T
203 
204 def hf_tableT(h_hat):
205     '''
206     :param h_hat: enthalpy in J/mol also known as heat of
    formation
207     :return Temp: temperature in K
208     '''
209     f2_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
210     h_hat = h_hat/4184
211     idx = (f2_table['H*-H*_298']-h_hat).abs().idxmin()
212     h_hat_closest = f2_table['H*-H*_298'][idx]
213     T_close = f2_table['T'][idx]
214     if h_hat > h_hat_closest:
215         h_hat_upper = f2_table['H*-H*_298'][idx+1]
216         T_upper = f2_table['T'][idx+1]
217         T = T_close + (T_upper-T_close)/(h_hat_upper-
    h_hat_closest)*(h_hat-h_hat_closest)
218     else:
219         h_hat_lower = f2_table['H*-H*_298'][idx-1]
220         T_lower = f2_table['T'][idx-1]
221         T = T_close + (T_lower-T_close)/(h_hat_lower-
    h_hat_closest)*(h_hat-h_hat_closest)
222 
223     return T
224 
225 
226 def oh_table_g(Temp):
227     '''
228     :param Temp: temperature in K
229     :return g_hat: Gibbs free energy in J/mol
230     '''
231     oh_table = pd.read_csv(TABLES / 'OH_table.csv')
232 
233     idx = (oh_table['T']-Temp).abs().idxmin()
234     T_cloest = oh_table['T'][idx]
235     g_hat_close = oh_table['g_hat'][idx]
236     if Temp > T_cloest:
237         T_upper = oh_table['T'][idx+1]
```

```
238         g_hat_upper = oh_table['g_hat'][idx+1]
239         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
240     else:
241         T_lower = oh_table['T'][idx-1]
242         g_hat_lower = oh_table['g_hat'][idx-1]
243         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
244 
245     return g_hat
246 
247 def h_table_g(Temp):
248     '''
249     :param Temp: temperature in K
250     :return g_hat: Gibbs free energy in J/mol
251     '''
252     h_table = pd.read_csv(TABLES / 'H_table.csv')
253 
254     idx = (h_table['T']-Temp).abs().idxmin()
255     T_cloest = h_table['T'][idx]
256     g_hat_close = h_table['g_hat'][idx]
257     if Temp > T_cloest:
258         T_upper = h_table['T'][idx+1]
259         g_hat_upper = h_table['g_hat'][idx+1]
260         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
261     else:
262         T_lower = h_table['T'][idx-1]
263         g_hat_lower = h_table['g_hat'][idx-1]
264         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
265 
266     return g_hat
267 
268 def h2o_table_g(Temp):
269     '''
270     :param Temp: temperature in K
271     :return g_hat: Gibbs free energy in J/mol
272     '''
273     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
274 
275     idx = (h2o_table['T']-Temp).abs().idxmin()
276     T_cloest = h2o_table['T'][idx]
277     g_hat_close = h2o_table['g_hat'][idx]
278     if Temp > T_cloest:
```

```
279         T_upper = h2o_table['T'][idx+1]
280         g_hat_upper = h2o_table['g_hat'][idx+1]
281         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
282     else:
283         T_lower = h2o_table['T'][idx-1]
284         g_hat_lower = h2o_table['g_hat'][idx-1]
285         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
286 
287     return g_hat
288 
289 def h2_table_g(Temp):
290     '''
291     :param Temp: temperature in K
292     :return g_hat: Gibbs free energy in J/mol
293     '''
294     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
295 
296     idx = (h2_table['T']-Temp).abs().idxmin()
297     T_cloest = h2_table['T'][idx]
298     g_hat_close = h2_table['g_hat'][idx]
299     if Temp > T_cloest:
300         T_upper = h2_table['T'][idx+1]
301         g_hat_upper = h2_table['g_hat'][idx+1]
302         g_hat = g_hat_close + (g_hat_upper-g_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
303     else:
304         T_lower = h2_table['T'][idx-1]
305         g_hat_lower = h2_table['g_hat'][idx-1]
306         g_hat = g_hat_close + (g_hat_lower-g_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
307 
308     return g_hat
309 
310 def h2o_table_cp(Temp):
311     '''
312     :param Temp: temperature in K
313     :return cp_hat: heat capacity in J/mol-K
314     '''
315     h2o_table = pd.read_csv(TABLES / 'H2O_table.csv')
316 
317     idx = (h2o_table['T']-Temp).abs().idxmin()
318     T_cloest = h2o_table['T'][idx]
319     cp_hat_close = h2o_table['cp_hat'][idx]
```

```
    if Temp > T_cloest:
        T_upper = h2o_table['T'][idx+1]
        cp_hat_upper = h2o_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h2o_table['T'][idx-1]
        cp_hat_lower = h2o_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def h2_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')

    idx = (h2_table['T']-Temp).abs().idxmin()
    T_cloest = h2_table['T'][idx]
    cp_hat_close = h2_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = h2_table['T'][idx+1]
        cp_hat_upper = h2_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h2_table['T'][idx-1]
        cp_hat_lower = h2_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def oh_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    oh_table = pd.read_csv(TABLES / 'OH_table.csv')

    idx = (oh_table['T']-Temp).abs().idxmin()
    T_cloest = oh_table['T'][idx]
```

```
    cp_hat_close = oh_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = oh_table['T'][idx+1]
        cp_hat_upper = oh_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = oh_table['T'][idx-1]
        cp_hat_lower = oh_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def h_table_cp(Temp):
    '''
    :param Temp: temperature in K
    :return cp_hat: heat capacity in J/mol-K
    '''
    h_table = pd.read_csv(TABLES / 'H_table.csv')

    idx = (h_table['T']-Temp).abs().idxmin()
    T_cloest = h_table['T'][idx]
    cp_hat_close = h_table['cp_hat'][idx]
    if Temp > T_cloest:
        T_upper = h_table['T'][idx+1]
        cp_hat_upper = h_table['cp_hat'][idx+1]
        cp_hat = cp_hat_close + (cp_hat_upper-cp_hat_close)/(T_upper-T_cloest)*(Temp-T_cloest)
    else:
        T_lower = h_table['T'][idx-1]
        cp_hat_lower = h_table['cp_hat'][idx-1]
        cp_hat = cp_hat_close + (cp_hat_lower-cp_hat_close)/(T_lower-T_cloest)*(Temp-T_cloest)

    return cp_hat

def MachSolve(gamma, r_A, guess):
    '''
    Solve for Mach number given gamma, r_A, and guess
    :param gamma: the gamma of the gas
    :param r_A: the ratio of the areas
    :param guess: the initial guess of the Mach number
    :return: Mach number
    '''
```

```
402
403     # M = var('M')
404     front_cont = ((gamma+1)/2)**(-(gamma+1)/(2*(gamma-1)))
405     sec_cont = (gamma-1)/2
406     func = lambda M: (((front_cont)*((1+sec_cont*M**2)**((
    gamma+1)/(2*(gamma-1)))))/M) - r_A
407     sol = fsolve(func, guess)
408
409     return sol
410
411 def h2_tableS(Temp):
412     '''
413     :param Temp: temperature in K
414     :return S_hat: entropy in J/mol-K
415     '''
416     h2_table = pd.read_csv(TABLES / 'Hydrogen_H2_table.csv')
417
418     idx = (h2_table['T']-Temp).abs().idxmin()
419     T_cloest = h2_table['T'][idx]
420     S_hat_close = h2_table['S'][idx]
421     if Temp > T_cloest:
422         T_upper = h2_table['T'][idx+1]
423         S_hat_upper = h2_table['S'][idx+1]
424         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
425     else:
426         T_lower = h2_table['T'][idx-1]
427         S_hat_lower = h2_table['S'][idx-1]
428         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
429
430     return S_hat
431
432 def hf_tableS(Temp):
433     '''
434     :param Temp: temperature in K
435     :return S_hat: entropy in J/mol-K
436     '''
437     hf_table = pd.read_csv(TABLES / '
    HydrogenFluoride_HF_table.csv')
438
439     idx = (hf_table['T']-Temp).abs().idxmin()
440     T_cloest = hf_table['T'][idx]
441     S_hat_close = hf_table['S'][idx]
442     if Temp > T_cloest:
```

```
443         T_upper = hf_table['T'][idx+1]
444         S_hat_upper = hf_table['S'][idx+1]
445         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
446     else:
447         T_lower = hf_table['T'][idx-1]
448         S_hat_lower = hf_table['S'][idx-1]
449         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
450 
451     return S_hat
452 
453 def f2_tableS(Temp):
454     '''
455     :param Temp: temperature in K
456     :return S_hat: entropy in J/mol-K
457     '''
458     f2_table = pd.read_csv(TABLES / 'Fluorine_F2_table.csv')
459 
460     idx = (f2_table['T']-Temp).abs().idxmin()
461     T_cloest = f2_table['T'][idx]
462     S_hat_close = f2_table['S'][idx]
463     if Temp > T_cloest:
464         T_upper = f2_table['T'][idx+1]
465         S_hat_upper = f2_table['S'][idx+1]
466         S_hat = S_hat_close + (S_hat_upper-S_hat_close)/(
    T_upper-T_cloest)*(Temp-T_cloest)
467     else:
468         T_lower = f2_table['T'][idx-1]
469         S_hat_lower = f2_table['S'][idx-1]
470         S_hat = S_hat_close + (S_hat_lower-S_hat_close)/(
    T_lower-T_cloest)*(Temp-T_cloest)
471 
472     return S_hat
473 # Me = MachSolve(1.237115, 77.5426, 5)
474 # print('Me = ', Me)
```